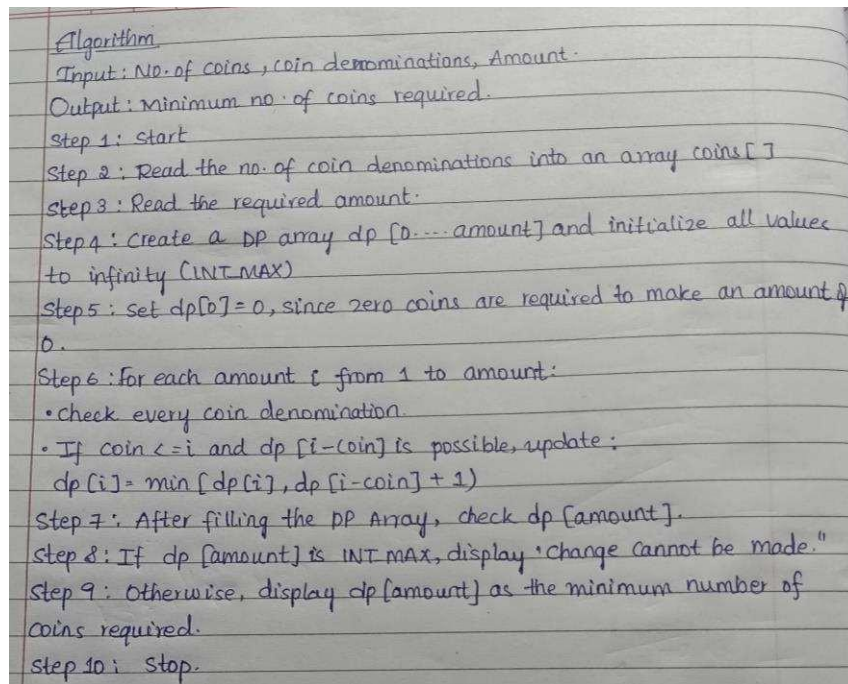


PRACTICAL -7

Aim: Implementation of making a change problem using dynamic programming

Algorithm:



Algorithm
Input: No. of coins, coin denominations, Amount.
Output: minimum no. of coins required.
Step 1: Start
Step 2: Read the no. of coin denominations into an array coins[]
Step 3: Read the required amount.
Step 4: Create a DP array dp [0....amount] and initialize all values to infinity (INT_MAX)
Step 5: Set dp[0] = 0, since zero coins are required to make an amount of 0.
Step 6: For each amount i from 1 to amount:
• Check every coin denomination
• If coin ≤ i and dp[i-coin] is possible, update:
dp[i] = min(dp[i], dp[i-coin] + 1)
Step 7: After filling the DP Array, check dp[amount].
Step 8: If dp[amount] is INT_MAX, display "change cannot be made."
Step 9: Otherwise, display dp[amount] as the minimum number of coins required.
Step 10: Stop.

program:

```
#include <iostream>

#include <vector>

#include <limits>

using namespace std;

int main() {

    int n, amount;

    cout << "Enter number of coins: ";

    cin >> n;
```

```
vector<int> coins(n);

cout << "Enter coin denominations: ";
for (int i = 0; i < n; i++) {
    cin >> coins[i];
}

cout << "Enter amount: ";
cin >> amount;

vector<int> dp(amount + 1, INT_MAX);

dp[0] = 0;

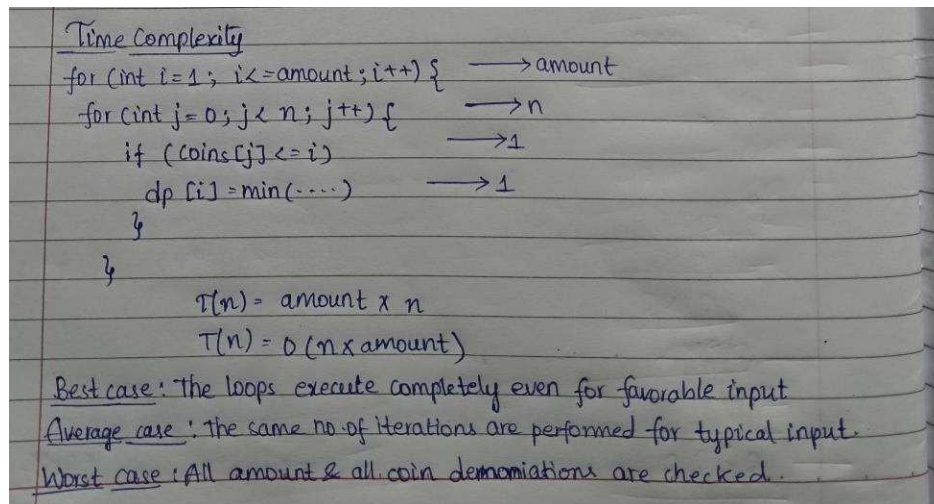
for (int i = 1; i <= amount; i++) {
    for (int j = 0; j < n; j++) {
        if (coins[j] <= i && dp[i - coins[j]] != INT_MAX) {
            dp[i] = min(dp[i], dp[i - coins[j]] + 1);
        }
    }
}
```

```
}  
  
if (dp[amount] == INT_MAX) {  
    cout << "Change cannot be made." << endl;  
} else {  
    cout << "Minimum number of coins required: "  
        << dp[amount] << endl;  
}  
  
return 0;  
}
```

Output:

```
Enter number of coins: 10  
Enter coin denominations: 5  
2  
1  
6  
5  
8  
7  
6  
3  
5  
Enter amount: 12  
Minimum number of coins required: 2
```

Time complexity:



Conclusion: Thus, the Making Change problem was successfully implemented using the Dynamic Programming technique. The algorithm efficiently finds the minimum number of coins required to make a given amount by breaking the problem into smaller subproblems and storing their results. This approach avoids repeated calculations and improves the efficiency compared to a simple recursive solution.